

Server architecture basics

COMPLETE LEARNING GUIDE

What You'll Learn

How a backend codebase is organized determines whether adding a feature six months from now takes an hour or a day. This guide covers layered architecture — separating routes, controllers, business logic (services), and data access into distinct layers with clear responsibilities — the classic MVC pattern as applied to backend APIs (as opposed to server-rendered pages), the trade-offs between a monolith and microservices, why statelessness is what actually enables horizontal scaling, and the supporting practices (configuration management, structured logging) that make a service operable in production rather than just functional on a laptop. The throughline is separation of concerns: each layer should be replaceable and independently testable without dragging the others along with it.

1. Layered Architecture: Routes, Controllers, Services, Data Access

A common and effective structure splits a backend into four layers with a strict one-directional dependency: routes map an HTTP method+path to a controller function; controllers translate an HTTP request into a call to business logic and translate the result back into an HTTP response (status code, JSON shape) — they should contain no actual business rules; services hold the actual business logic, independent of HTTP entirely (so they're callable from a route, a background job, or a test with no framework involved); and the data access layer (repositories, or an ORM directly) is the only layer that talks to the database. A route handler that directly runs a SQL query and does business logic in the same function collapses all four layers into one, which is fast to write initially and painful to test or change later.

JAVASCRIPT

```
// routes/students.js
router.post("/students", asyncHandler(studentController.create));

// controllers/studentController.js
async function create(req, res) {
  const student = await studentService.registerStudent(req.body);
  res.status(201).json(student);
}

// services/studentService.js — no req/res, no HTTP knowledge
async function registerStudent(input) {
  if (await studentRepo.findByEmail(input.email)) {
    throw new ConflictError("Email already registered");
  }
}
```

```
return studentRepo.create(input);
```


2. MVC in a Backend/API Context

Model-View-Controller predates REST APIs and originally described server-rendered pages, where the View literally rendered HTML. In a JSON API, there's no traditional view template — the 'view' is effectively the JSON serialization of the response, and the Model is the data layer (often the ORM models directly). The Controller role maps closely onto the controller layer described above. Understanding this mapping matters because 'MVC' shows up constantly in framework documentation and job descriptions, and knowing that the View concept has simply been replaced by response serialization in an API context avoids confusion when a framework's folder structure doesn't obviously match the classic diagram.

3. Monolith vs. Microservices

A monolith deploys the entire application (all features, all domains) as a single unit with a single codebase and typically a single database — simpler to develop, test, and deploy when a team and domain are still small, since there's no network call between 'services' that live in the same process. Microservices split the application into independently deployable services, each owning its own data, communicating over the network (HTTP/gRPC/message queues) — this buys independent scaling and deployment per service, at the direct cost of distributed-systems complexity: network failures, data consistency across services, and significantly more operational overhead. The common, well-supported advice is to start with a well-organized monolith and split out services only once a specific, demonstrated need (team scaling boundaries, wildly different scaling requirements between components) justifies the added complexity.

4. Statelessness and Horizontal Scaling

A stateless server keeps no request-specific data in its own memory between requests — anything that needs to persist (session data, uploaded files, cached values) lives in a shared external store (a database, Redis, object storage) that every server instance can access equally. This is what makes horizontal scaling (running multiple identical server instances behind a load balancer) actually work: a load balancer can route any request to any instance interchangeably, because no instance holds state that only it knows about. Storing session data in a server's local memory is the classic violation — it works fine with one server instance and breaks unpredictably the moment a second instance is added, since a user's session might only exist on the instance that handled their login.

5. Configuration and Dependency Injection

Configuration (database URLs, API keys, feature flags, timeouts) should be injected into the application at startup — typically from environment variables — rather than hardcoded, so the exact same codebase runs correctly across development, staging, and production by changing only its configuration, never its code. Dependency injection (passing a service's dependencies, like a database client, into it explicitly rather than having it construct or import them directly) is what makes services testable in isolation — a unit test can inject a fake/mock database client instead of requiring a real database connection just to test business logic.

6. Logging and Observability Basics

Scattered `console.log` statements are adequate for local development but inadequate in production, where you need to search, filter, and correlate logs across potentially many server instances handling concurrent requests. Structured logging (emitting logs as JSON with consistent fields — timestamp, request ID, severity level, message) rather than free-form text makes logs queryable in a log aggregation tool. Attaching a unique request ID to each incoming request (generated at the very first middleware and threaded through every subsequent log line for that request) is what lets you reconstruct the full story of one specific request across multiple log lines and, in a microservices setup, across multiple services.

7. Separating Concerns for Testability

The entire point of the layered structure discussed above is testability: business logic in the service layer, with no HTTP or database code mixed in, can be unit-tested by calling functions directly with plain JavaScript objects and mock dependencies — no test server, no test database, no network involved, making these tests fast and reliable. Controllers and routes are thin enough that they're usually covered by a smaller number of integration tests that verify the HTTP contract (status codes, response shape) rather than re-testing business rules already covered at the service layer. Tangling business logic directly into route handlers forces every test to go through the full HTTP + database stack just to verify a single business rule, making the test suite slow and brittle.

Common Pitfalls

- Writing business logic directly inside route handlers, making it untestable without spinning up a full HTTP server and database.
- Storing session or other per-user state in a server instance's local memory, which silently breaks once a second instance is added behind a load balancer.
- Adopting a microservices architecture before the team or domain actually needs the independent scaling/deployment it trades complexity for.
- Hardcoding configuration values (URLs, keys, timeouts) directly in code instead of injecting them via environment variables.
- Tight coupling between layers (e.g. a service function directly constructing its own database client) that makes unit testing require a real database connection.
- Using unstructured `console.log` statements in production instead of structured, queryable logs with request-level correlation IDs.
- Letting circular dependencies form between modules/layers, making the codebase's actual dependency graph hard to reason about.

Key Takeaways

- Layered architecture (routes !' controllers !' services !' data access) keeps business logic independently testable and framework-agnostic.
- In an API context, MVC's 'View' is effectively the JSON response shape, not an HTML template.
- Start with a well-organized monolith; split into microservices only once a specific, demonstrated need justifies the added distributed-systems complexity.
- Statelessness — no per-user data held in server memory — is the prerequisite for horizontal scaling to work at all.
- Inject configuration via environment variables so the same code runs correctly across every environment.
- Structured logging with request-correlation IDs is what makes production issues debuggable across concurrent requests and multiple instances.

Learn More

Martin Fowler — Microservices

<https://martinfowler.com/articles/microservices.html>

The Twelve-Factor App

<https://12factor.net/>

Node.js Best Practices (GitHub)

<https://github.com/goldbergonyi/nodebestpractices>